

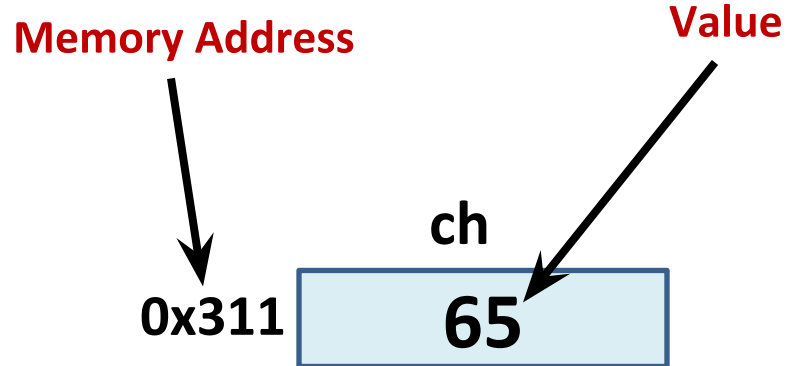
Content

- Pointers
 - Dynamic memory allocation
 - Using pointers
 - Const vs non-const pointer
 - Pointer to const vs non-const data
 - Array of pointers
 - Memory layout of c++ program
-

Introduction to pointers

- When we **declare** a **variable**, some **memory** is **allocated** for it.
- Thus, we have **two properties** for any **variable**:
 1. Its **Address**
 2. and its **Data value**

E.g., `char ch = 'A';`



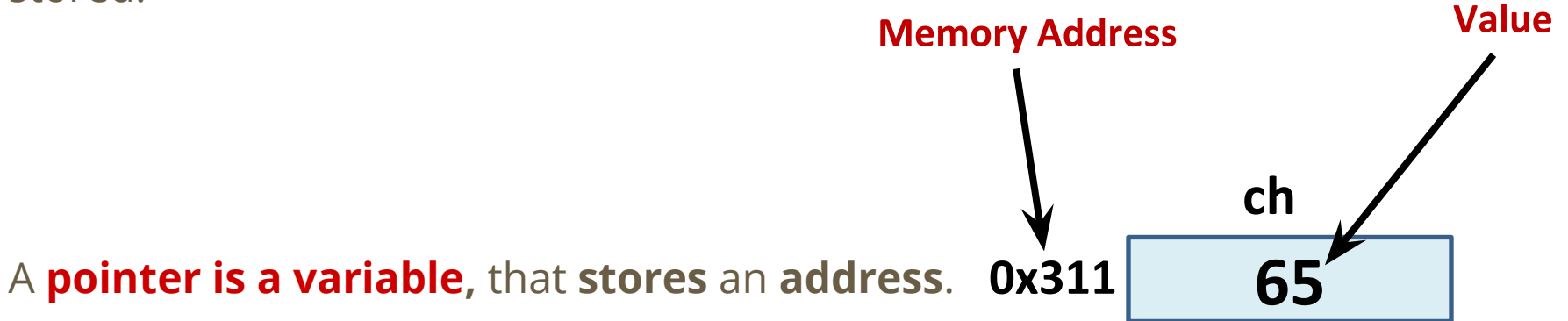
Introduction to pointers

How to access the address of a variable?

Address of a variable can be accessed through the referencing operator “&”

```
char ch = 'A';
```

Example: **&ch** will return **memory location** where the data value for “**ch**” is stored.



Introduction to pointers

Pointer is often called point to variable because this can store the address of a variable.

Syntax:

Type* <pointer variable Name>;

Sample:

```
int* ptr;
```

creates a ***pointer variable*** named "**ptr**", that will ***store address*** (memory location) of some **int type** variable.

Pointers Data-Type

● Question:

Why is it important to declare the type of the variable that a pointer points to?

Aren't all memory addresses of the same length?

Pointers Type

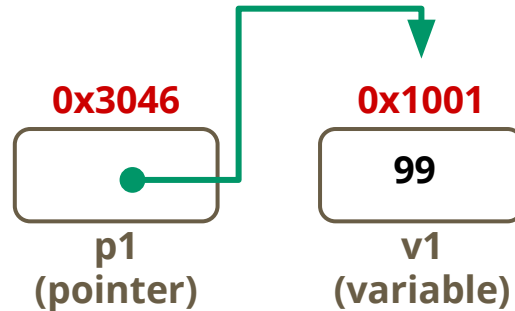
- Answer:

- **All memory addresses are of the same length,**
 - However, with operation “p++” where “p” is a **pointer** □ the **compiler needs to know** the **data type** of the **variable “p”** (**to jump at next memory location**)
 - Examples:
 - If “p” is a **character-pointer** then “p++” will increment “p” by **one byte** (next location)
 - if “p” is an **integer-pointer** its value on “p++” would be incremented by **4 bytes** (next loc.)

Initializing a pointer

As discussed earlier “&” is used to access the address of a variable. This address can then be assigned to the **pointer variable**.

```
int* p1;  
int v1=99;  
p1 = &v1;
```



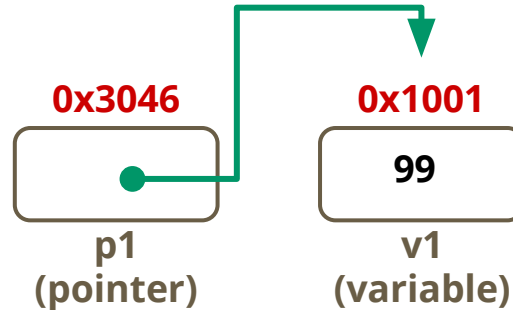
Now **p1** stores to the memory location, where v1 is stored.

- v1 is an integer variable
- P1 is a integer pointer (i.e. points to an integer memory location)

Accessing pointer's value

Data value of a pointer can simply be accessed by its name.

```
int* p1;  
int v1=99;  
p1 = &v1;  
cout<< p1;
```



We can also access the value stored on the address **0x1001** by using "*****" operator. This is called **dereferencing**.

***p1** □ ***0x1001** □ 99

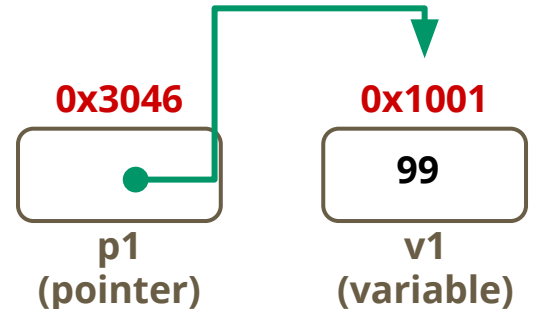
Dereferencing a pointer

When we are using ***** operator in-front of a pointer, such as ***p1** this implies:

"The variable values pointed to by p1"

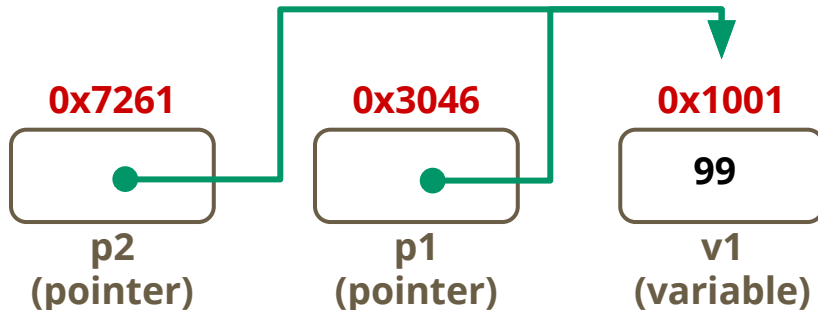
Here the ***** is the **dereferencing operator** and p1 is said to be dereferenced.

```
int v1=99;  
int* p1= &v1;  
cout<<" P1 points to the value: "<<*p1;  
cout<<" P1 points to the variable: "<<p1;
```



Pointer Assignment

- **Assignment operator (=)** is used to assign value of one **pointer** to another
- **Pointer stores addresses** so **p1=p2** copies **an address value** into another pointer



```
1  #include <iostream>
2
3  using namespace std;
4
5  int main()
6  {
7      int v1 = 55;
8      int* p1 = &v1;
9      int* p2;
10     p2=p1;
11     cout << "Value:" << *p1 << endl;
12     cout << "Value:" << *p2 << endl;
13
14     cout << "Address stored: " << p1 << endl;
15     cout << "Address stored:" << p2 << endl;
16
17     return 0;
18 }
```

Using pointer to assign a value to a variable

```
9  #include <iostream>
10
11  using namespace std;
12
13  int main()
14  {
15      int a;
16      int* ptr=&a;
17
18      *ptr=9;
19      cout<<a;
20
21      return 0;
22  }
```

Memory Management

- C++ leaves memory management mostly up to the programmer:

Pros: write programs that use memory very efficiently

Cons: write programs that waste memory or do not work at all

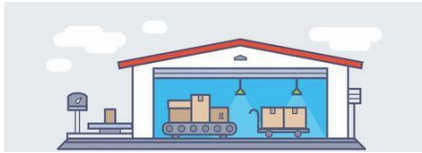


For efficient program working, we need good understanding of the memory models

Memory Management

- Most of the time the amount of space required is unknown at compile time.

If we want to store data, how much space do we require?



- Too little, and we might run out of space.
- Too much, and we waste a lot of space.

Dynamic Memory Allocation

- Used when space requirements are unknown at compile time
- **Dynamic Memory Allocation (DMA):-**
 - With Dynamic memory allocation we can allocate/deletes memory (elements of an array) at runtime or execution time.

✗ Common errors caused by poor memory management:

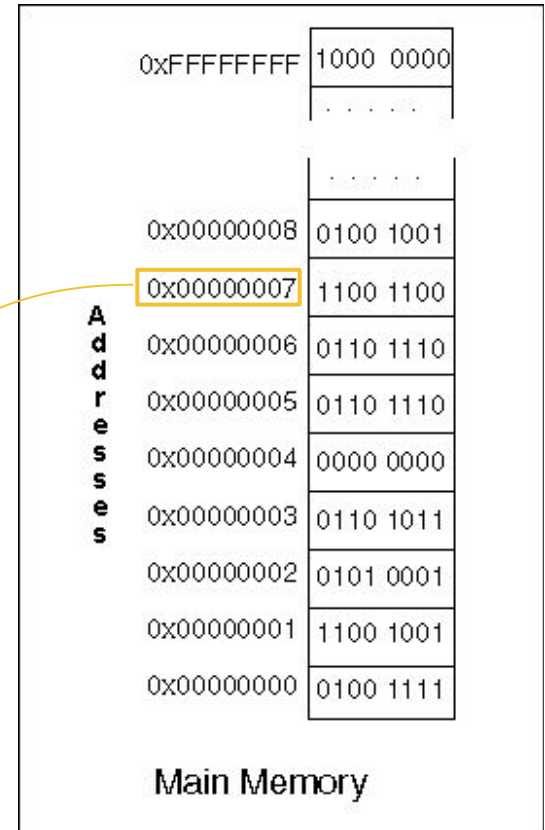
- Using a variable before it has been initialized
- **Allocating memory for storage and not deleting it**
- Using a value after it has been deleted

Dynamic Memory Allocation

Allocating Memory:

The **new** keyword is used to dynamically allocate memory. This request for memory allocation on the Free Store. Return of this keyword is either *exception* or an *address* to the allocated memory.

Dynamically allocated memory cannot have a "name", it must be referred to.



Allocation

Syntax:

data-type* pointer-variable = **new** data-type; //allocate memory for 1 element

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int *p1 = NULL;
7      p1 = new int;
8
9      int *p2 = new int;
10
11     char *p3;
12     p3 = new char;
13
14
15     return 0;
16 }
17
```

Allocation

Syntax:

data-type* pointer-variable = **new** data-type[size]; //allocate block of memory

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int *p1 = NULL;
7      p1 = new int[10];
8
9      int *p2 = new int[10];
10
11     char *p3;
12     p3= new char[10];
13
14
15     return 0;
16 }
17
```

What happens if there is not enough memory?

When new operator request for the memory then if there is a free memory is available then it returns valid address either it throws bad_alloc exception.

To avoid the exception throw we can use “nothrow” with the new operator. When we are used “nothrow” with new operator, it will either return a valid address or return 0 (NULL).

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int *p = new(nothrow) int;
7      if (!p)
8      {
9          cout << "Memory allocation failed\n";
10     }
11
12     return 0;
13 }
14
```

Initialization

data-type pointer-variable = **new** data-type(value);

data-type pointer-variable = **new** data-type[size]{ elements separated by ","};

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int *p = new int(25);
7      float *q = new float(75.25);
8      char *c = new char[5]{ 'a', 'b', 'c', 'd', 'e' };
9
10     return 0;
11 }
12
```

Deallocation

delete pointer-variable; //deallocation single variable memory

delete[] pointer-variable; // deallocation block of memory

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int *p = new int(25);
7      float *q = new float;
8      char *c = new char[5]{ 'a', 'b', 'c', 'd', 'e' };
9
10     delete p;
11     delete q;
12
13     delete[] c;
14
15     return 0;
16 }
```

Example

```
int main()
{
    int *p;

    p = new int;
    *p = 99;

    delete p;
    return 0;
}
```

A d d r e s s e s	0xFFFFFFFF	1000 0000
		
		
	0x00000008	0100 1001
	0x00000007	1100 1100
	0x00000006	0110 1110
	0x00000005	0110 1110
	0x00000004	0000 0000
	0x00000003	0110 1011
	0x00000002	0101 0001
	0x00000001	1100 1001
	0x00000000	0100 1111

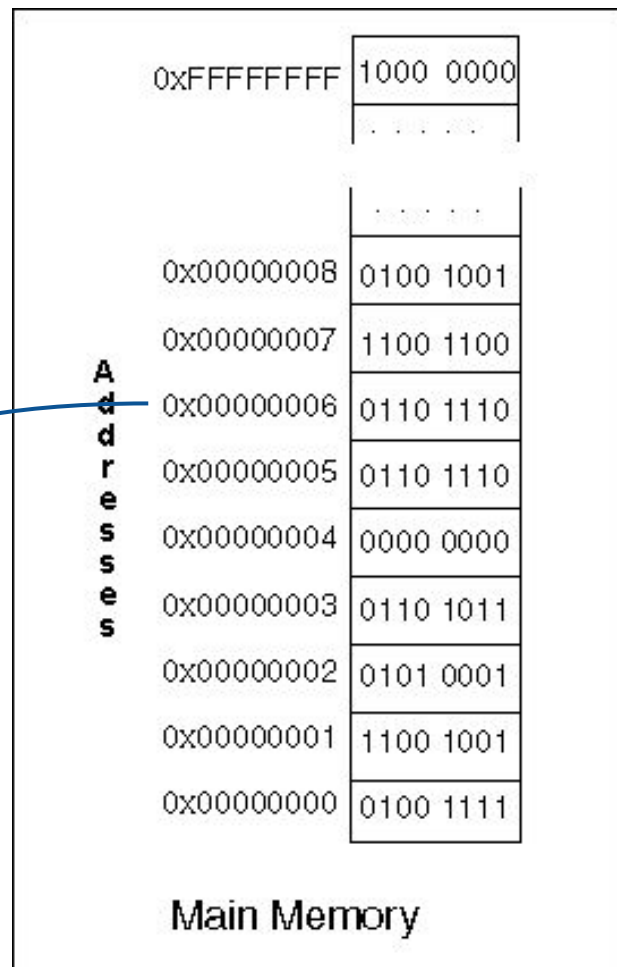
Main Memory

Example

```
int main()
{
    int *p;

    p = new int;
    *p = 99;

    delete p;
    return 0;
}
```

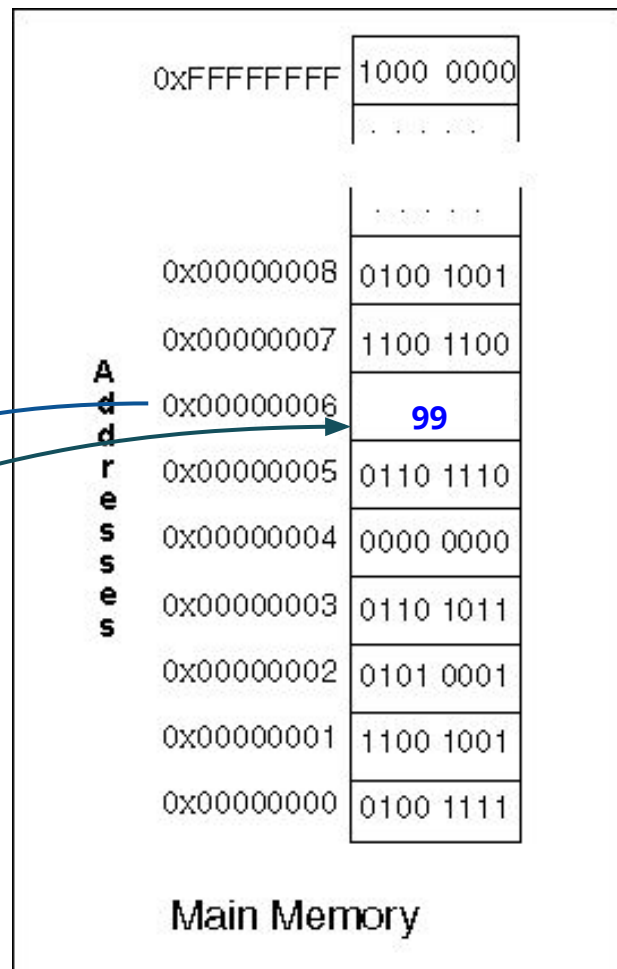


Example

```
int main()
{
    int *p;

    p = new int;
    *p = 99;

    delete p;
    return 0;
}
```

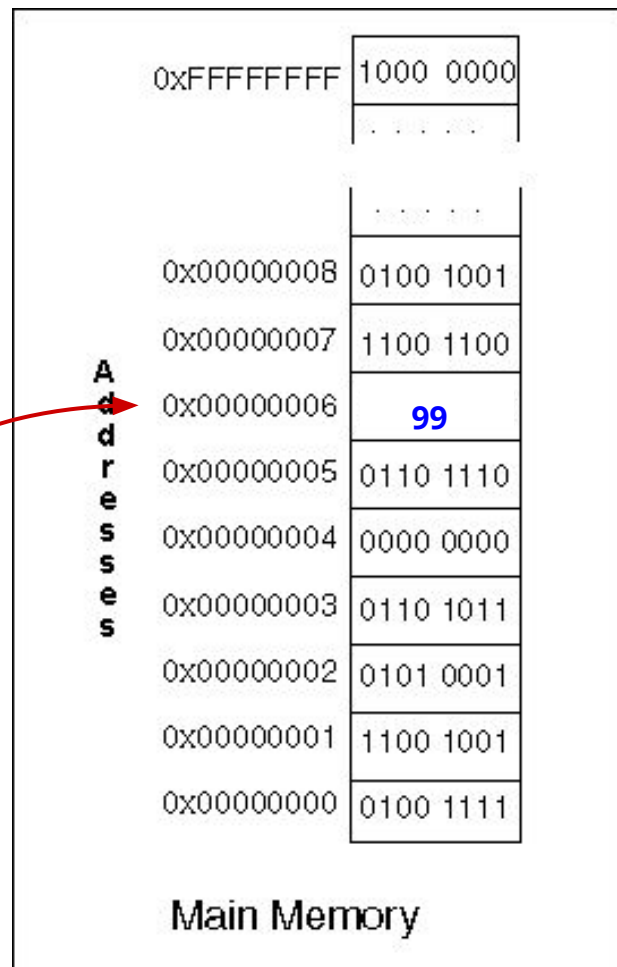


Example

```
int main()
{
    int *p;

    p = new int;
    *p = 99;

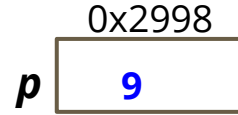
    delete p;
    return 0;
}
```



Aliasing

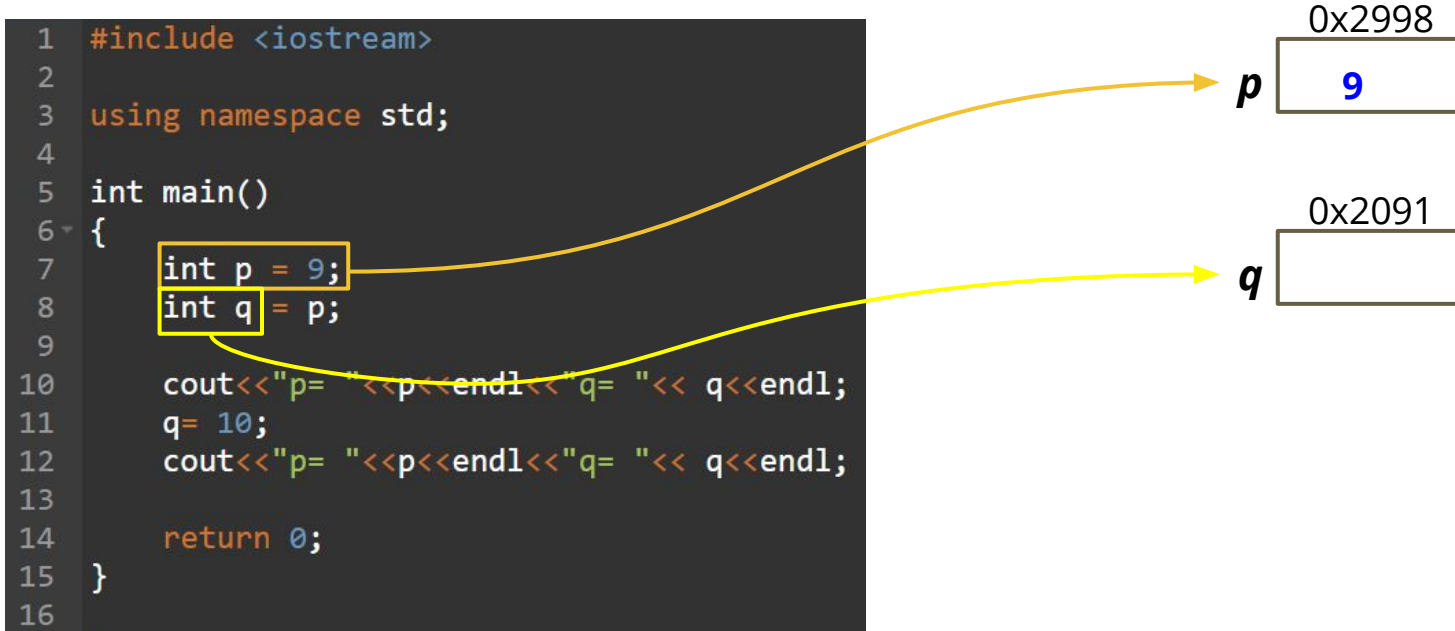
In computer programming, **aliasing** refers to the situation where the same memory location can be accessed using different names.

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main()
6  {
7      int p = 9;
8      int q = p;
9
10     cout<<"p= "<<p<<endl<<"q= "<< q<<endl;
11     q= 10;
12     cout<<"p= "<<p<<endl<<"q= "<< q<<endl;
13
14     return 0;
15 }
16
```



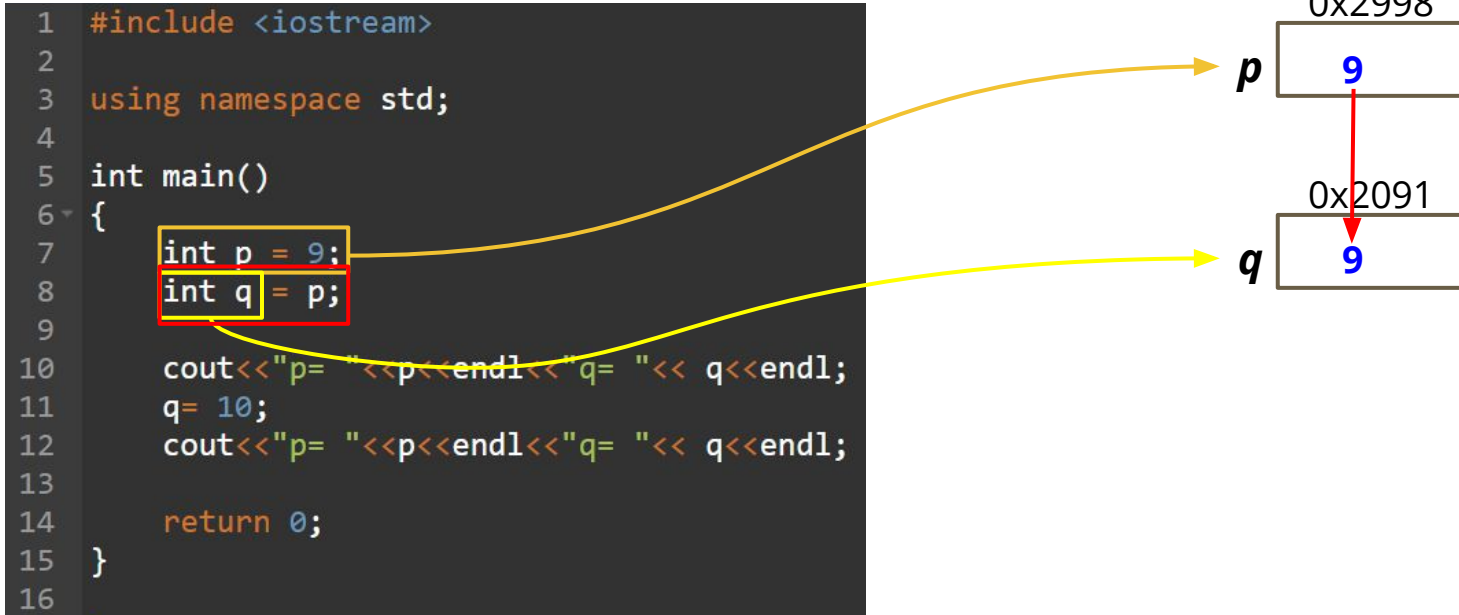
Aliasing

In computer programming, **aliasing** refers to the situation where the same memory location can be accessed using different names.



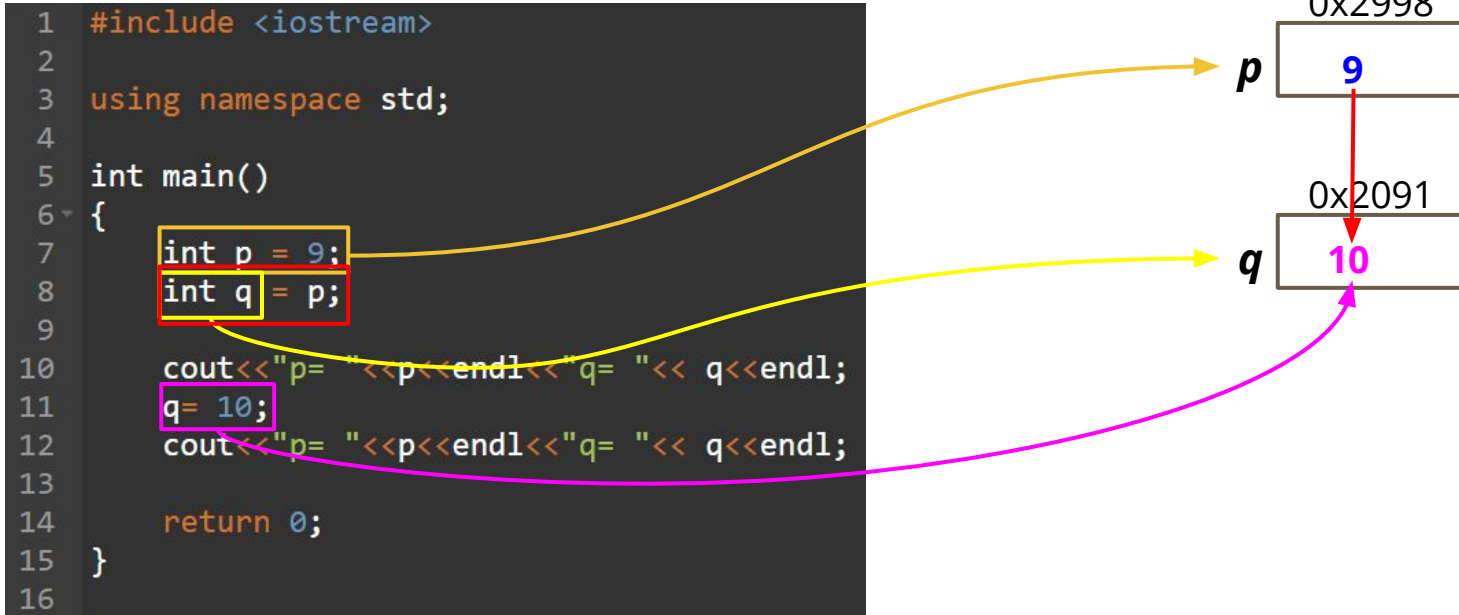
Aliasing

In computer programming, **aliasing** refers to the situation where the same memory location can be accessed using different names.



Aliasing

In computer programming, **aliasing** refers to the situation where the same memory location can be accessed using different names.



Aliasing

In computer programming, **aliasing** refers to the situation where the same memory location can be accessed using different names.

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main()
6  {
7      int *p = new (nothrow) int(9);
8      int *q = p;
9
10     cout<<"p= "<<*p<<endl<<"q= "<< *q<<endl;
11     *q= 10;
12     cout<<"p= "<<*p<<endl<<"q= "<< *q<<endl;
13
14     delete q;
15     return 0;
16 }
```

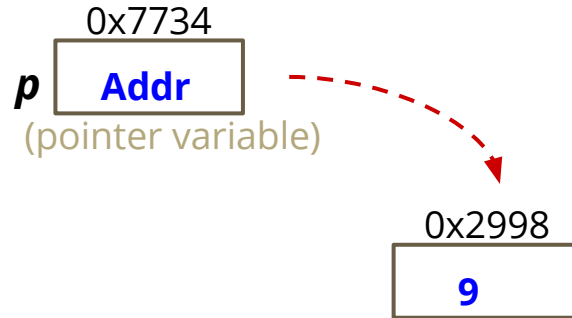
0x2998

9

Aliasing

In computer programming, **aliasing** refers to the situation where the same memory location can be accessed using different names.

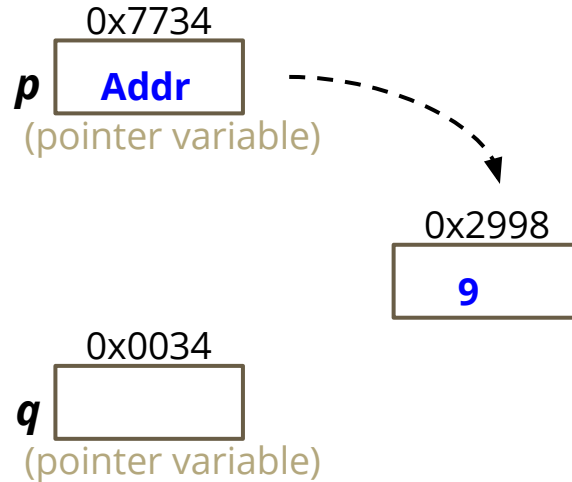
```
1  #include <iostream>
2
3  using namespace std;
4
5  int main()
6  {
7      int *p = new (nothrow) int(9);
8      int *q = p;
9
10     cout<<"p= "<<*p<<endl<<"q= "<< *q<<endl;
11     *q= 10;
12     cout<<"p= "<<*p<<endl<<"q= "<< *q<<endl;
13
14     delete q;
15     return 0;
16 }
```



Aliasing

In computer programming, **aliasing** refers to the situation where the same memory location can be accessed using different names.

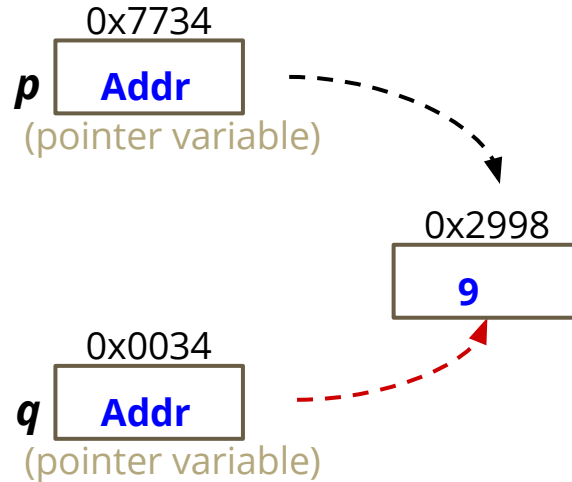
```
1  #include <iostream>
2
3  using namespace std;
4
5  int main()
6  {
7      int *p = new (nothrow) int(9);
8      int *q = p;
9
10     cout<<"p= "<<*p<<endl<<"q= "<< *q<<endl;
11     *q= 10;
12     cout<<"p= "<<*p<<endl<<"q= "<< *q<<endl;
13
14     delete q;
15     return 0;
16 }
```



Aliasing

In computer programming, **aliasing** refers to the situation where the same memory location can be accessed using different names.

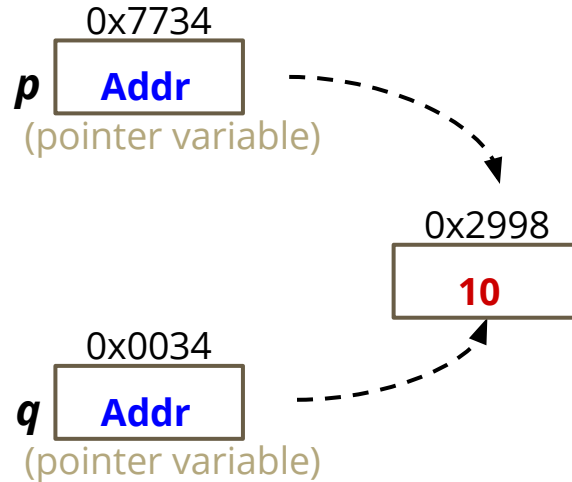
```
1  #include <iostream>
2
3  using namespace std;
4
5  int main()
6  {
7      int *p = new (nothrow) int(9);
8      int *q = p;
9
10     cout<<"p= "<<*p<<endl<<"q= "<< *q<<endl;
11     *q= 10;
12     cout<<"p= "<<*p<<endl<<"q= "<< *q<<endl;
13
14     delete q;
15     return 0;
16 }
```



Aliasing

In computer programming, **aliasing** refers to the situation where the same memory location can be accessed using different names.

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main()
6  {
7      int *p = new (nothrow) int(9);
8      int *q = p;
9
10     cout<<"p= "<<*p<<endl<<"q= "<< *q<<endl;
11     *q= 10;
12     cout<<"p= "<<*p<<endl<<"q= "<< *q<<endl;
13
14     delete q;
15     return 0;
16 }
```

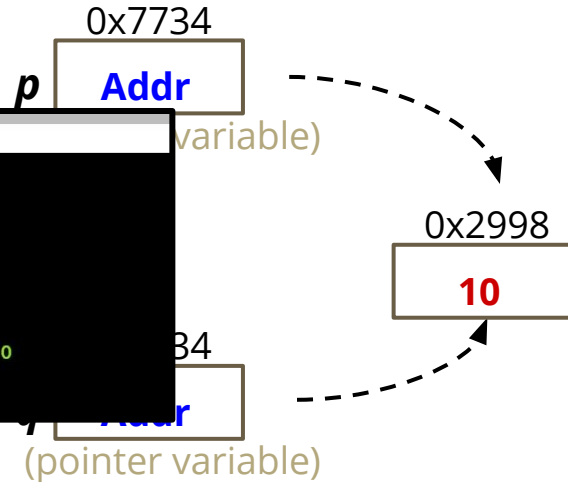


Aliasing

In computer programming, **aliasing** refers to the situation where the same memory location can be accessed using different names.

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main()
6  {
7      int *p = new (nothrow) int[10];
8      int *q = p;
9
10     cout<<"p= " <<*p<<endl;
11     *q= 10;
12     cout<<"p= " <<*p<<endl;
13
14     delete q;
15     return 0;
16 }
```

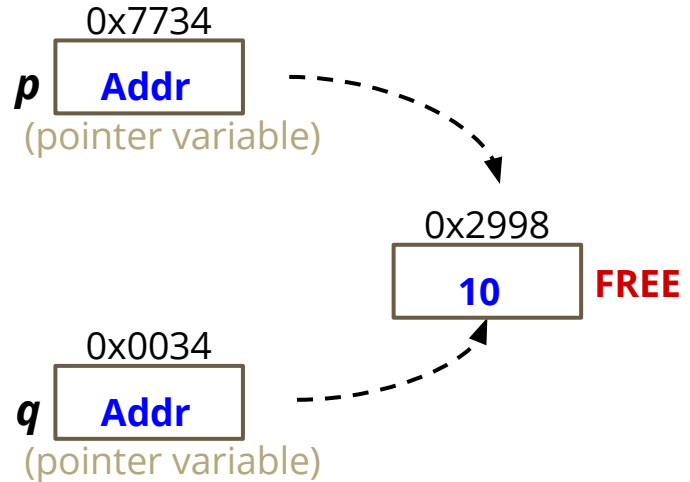
```
p= 9
q= 9
p= 10
q= 10
...Program finished with exit code 0
Press ENTER to exit console.
```



Aliasing

In computer programming, **aliasing** refers to the situation where the same memory location can be accessed using different names.

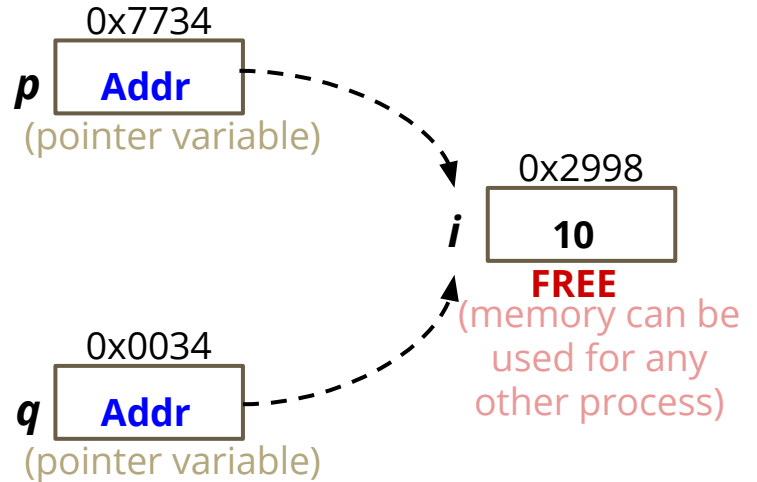
```
1  #include <iostream>
2
3  using namespace std;
4
5  int main()
6  {
7      int *p = new (nothrow) int(9);
8      int *q = p;
9
10     cout<<"p= "<<*p<<endl<<"q= "<< *q<<endl;
11     *q= 10;
12     cout<<"p= "<<*p<<endl<<"q= "<< *q<<endl;
13
14     delete q;
15     return 0;
16 }
```



Aliasing

In computer programming, **aliasing** refers to the situation where the same memory location can be accessed using different names.

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main()
6  {
7      int *p = new (nothrow) int(9);
8      int *q = p;
9
10     cout<<"p= "<<*p<<endl<<"q= "<< *q<<endl;
11     *q= 10;
12     cout<<"p= "<<*p<<endl<<"q= "<< *q<<endl;
13
14     delete q;
15     return 0;
16 }
```



What if we try to access pointer ***p*** and ***q***?

p and q are pointing to the same memory but that memory is being used by another process. Accessing this memory can give unexpected result. Such pointers are known as ***dangling pointers***.



The delete operator does not delete the pointer, it takes the memory being pointed to and returns it back.

How to avoid dangling pointers?

- **For Variables:**

```
delete v1;  
v1 = NULL;
```

- **For Arrays:**

```
delete[ ] arr;  
arr = NULL;
```



This is termed as undangling the pointers.

How to avoid dangling pointers?

- **For Variables:**

```
delete v1;  
v1 = NULL;
```

- **For Arrays:**

```
delete[ ] arr;  
arr = NULL;
```

- **Cannot Dereference a Pointer whose value is Null:**

ptr = 0; OR ptr=NULL;

cout << *ptr << endl; // ERROR: ptr

**// does not point to
// a valid address**

This is termed as undangling the pointers.



What's Wrong with the Following:

```
ptr = NULL;  
delete ptr;
```